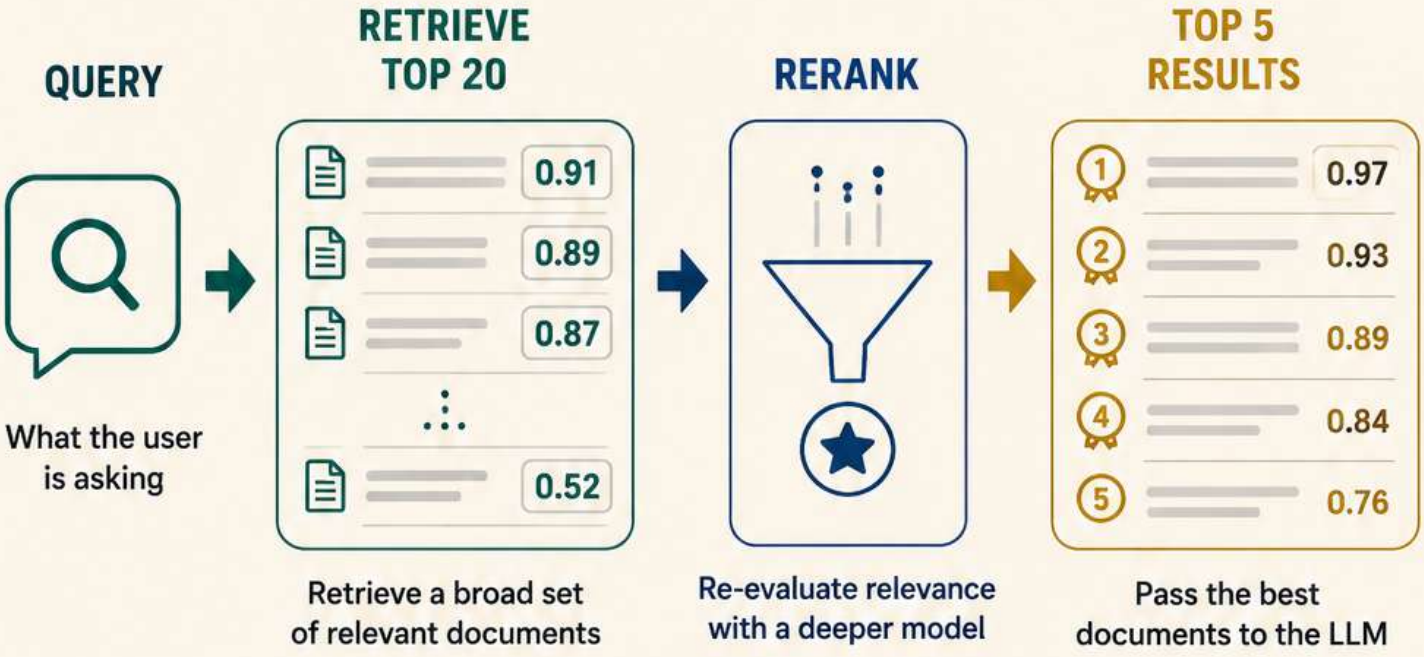
RERANKING IN RAG SYSTEMS

Why retrieving the right documents isn't always enough



Reranking turns good results into the RIGHT results.

VANSH BUDHIRAJA

THE PROBLEM WITH TOP-K RETRIEVAL

High retrieval score ≠ best answer context

QUERY

How does KV cache reduce LLM inference latency?

TOP 5 FROM RETRIEVER

(Vector / BM25 Retrieval)

01	**Doc A** KV cache overview	0.91
02	**Doc B** Attention mechanism basics	0.89
03	**Doc C** Transformer architecture intro	0.87
04	**Doc D** Caching in databases	0.85
05	**Doc E** Model inference optimization	0.84

THE ISSUE

Retriever is optimized for speed and recall, not precise relevance.

WHAT WE ACTUALLY NEED

Best context for the answer

01	**Doc C** Transformer architecture intro	(More relevant)
02	**Doc A** KV cache overview	(More relevant)
03	**Doc E** Model inference optimization	(More relevant)
04	**Doc B** Attention mechanism basics	(Less relevant)
05	**Doc D** Caching in databases	(Less relevant)

Top results may contain:

- Semantically close but not truly relevant
- Redundant or overlapping information
- Missing the most helpful context packets

RETRIEVAL FINDS CANDIDATES.
RERANKING FINDS THE RIGHT ONES.

WHAT DOES A RERANKER DO?

Reranking sits between retrieval and generation to pick the most relevant context.

USER QUERY
What the user is asking

Query
e.g. "How does KV cache reduce LLM latency?"

RETRIEVER
Fetch a broad set of documents

Initial Retrieval
Top 20–50 documents (Dense / Sparse / Hybrid)



RERANKER
Re-evaluate each document deeply

Reranker (Cross-Encoder)
Scores each document based on true relevance with the query



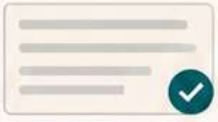
TOP RESULTS
Pass only the best context

Top 3–5 Documents
High relevance documents sent to the LLM



LLM
Generate accurate and grounded response

LLM
Uses the best context to generate the answer



KEY ROLE

Reranker doesn't just look at embeddings — it understands the query + document together.

WHY IT MATTERS

- Better relevance
- Reduce noise
- Higher accuracy
- Less hallucination

WHAT IT IMPROVES

- Answer quality
- Context precision
- User experience
- Knowledge grounding

WITHOUT RERANKING

0.91 → 0.89 → 0.87 → 0.85 → 0.52

High scores. But not the right order.

WITH RERANKING

0.97 → 0.94 → 0.89 → 0.78 → 0.61

Right scores. Right order. Better answers.

> Retrieve widely. Rerank deeply. Generate confidently.

BI-ENCODER vs CROSS-ENCODER

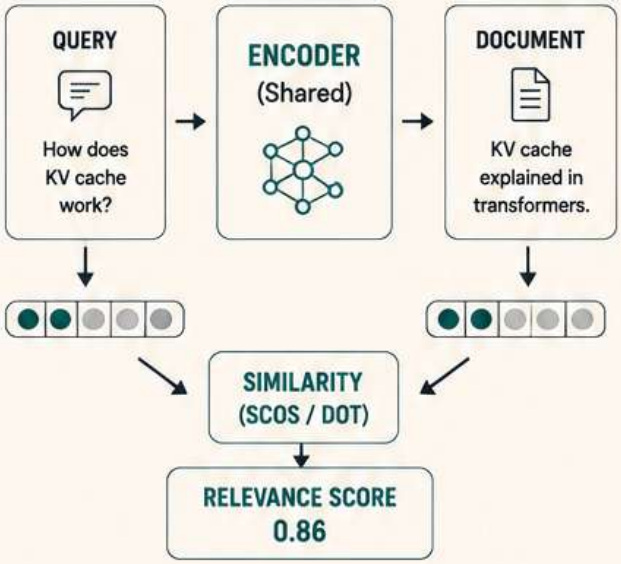
Two different ways to measure relevance.

BI-ENCODER
Fast & Scalable

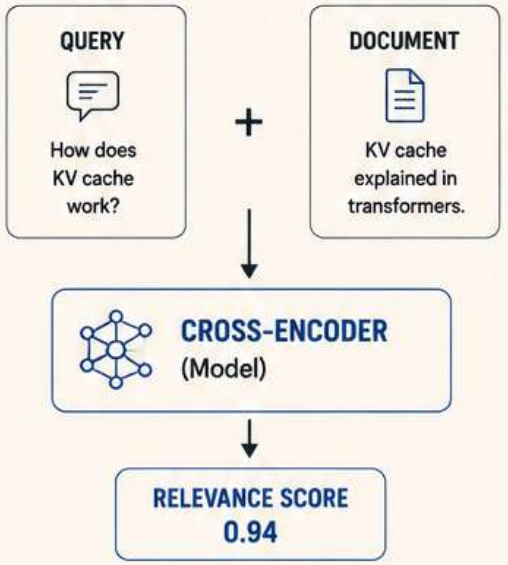
VS

CROSS-ENCODER
Accurate but Expensive

HOW IT WORKS



HOW IT WORKS



KEY POINTS

- Encodes query and documents separately.
- Uses similarity between embeddings.
- Very fast and scalable.
- Good for retrieving large candidate sets.

BEST FOR
First-stage retrieval when you have millions of documents.

KEY POINTS

- Takes query and document together as input.
- Understands full context and relationships.
- More accurate relevance judgment.
- Slower and computationally expensive.

BEST FOR
Reranking a small set of candidates (Top 20–50).

IN SIMPLE TERMS

BI-ENCODER
Finds potentially relevant documents quickly.

→

CROSS-ENCODER
Finds the most relevant documents accurately.

RETRIEVER = SPEED & RECALL.
RERANKER = ACCURACY & PRECISION.

RERANKING EXAMPLE

Same query. Different order. Much better context.

QUERY:
"How does KV cache reduce LLM inference latency?"

INITIAL RETRIEVAL

Top 5 from Retriever (Vector/BM25)

01	Doc A — KV cache overview and basics	0.92
02	Doc B — Attention mechanism in transformers	0.89
03	Doc C — Transformer inference optimization	0.87
04	Doc D — Caching in databases explained	0.84
05	Doc E — Model parallelism techniques	0.81

AFTER RERANKING

Top 5 after Cross-Encoder Reranker

01	Doc C — Transformer inference optimization	0.97
02	Doc A — KV cache overview and basics	0.94
03	Doc D — Caching in databases explained	0.78
04	Doc B — Attention mechanism in transformers	0.61
05	Doc E — Model parallelism techniques	0.47

WHAT CHANGED?

The reranker understood deeper relevance to the query, moved truly helpful documents up, and pushed weak or loosely related ones down.

RERANKING IMPROVES QUALITY, NOT JUST THE SCORE.

HIGHER RELEVANCE → BETTER CONTEXT → BETTER ANSWERS

WHEN SHOULD YOU USE RERANKING?

Use reranking when **relevance** matters more than **speed**.

1. Large candidate set?
Retriever returns many documents.

NO → SKIP RERANKING

YES ↓

2. Results are noisy or mixed in quality?
Many irrelevant or redundant documents appear.

NO → SKIP RERANKING

YES ↓

3. Is accuracy more important than retrieval latency?
You can afford extra computation for better relevance.

NO → SKIP RERANKING

YES ↓

USE RERANKING
Rerank and pass the best 3–5 documents to the LLM.

SKIP RERANKING
Send top results directly to the LLM.

BEST FOR

COMPLEX QUERIES
Queries with nuance, intent, or detailed requirements.

SIMILAR DOCUMENTS
Many documents look alike. Reranker finds the subtle differences.

NOISY RETRIEVAL
Initial retrieval returns irrelevant or off-topic candidates.

HIGH-VALUE RAG
When quality of context impacts the quality of the final answer.

MULTI-SOURCE KBs
Useful across multiple databases, domains, or knowledge bases.

KEY IDEA
Reranking adds extra compute, but it dramatically improves the **quality of context** sent to the LLM.

USE RERANKING STRATEGICALLY.
BETTER CONTEXT. BETTER ANSWERS.

THE COMPLETE RAG RETRIEVAL STACK

From query to answer – where reranking fits.

1. **USER QUERY**
   Natural language question or request
   → User asks a question.
2. **QUERY PROCESSING**
   Rewrite, expand, or transform the query for better retrieval
   → Improve the query to increase recall and relevance.
3. **DENSE + SPARSE RETRIEVAL**
   Retrieve broadly from vector DB and/or keyword index (BM25)
   → Get a wide set of potentially relevant documents (recall).
4. **TOP 20–50 CANDIDATES**
   Initial candidate pool from retrievers
   → Large enough set to not miss the right documents.
5. **DERANKER (CROSS-ENCODER)**
   Re-rank candidates by deep relevance to the query
   → Finds the most relevant documents, not just the most similar ones.
6. **TOP 3–5 CONTEXT**
   Best and most relevant documents selected
   → Smaller, high-quality set passed to the LLM (precision).
7. **CONTEXT ASSEMBLY**
   Combine, format, and prepare context for the LLM
   → Structure the context for maximum clarity and effectiveness.
8. **LLM**
   Generate answer using high-quality context
   → LLM produces accurate, grounded, and helpful answers.
9. **ANSWER**
   Final response to the user
   → Better context → better answers.

WHY THIS STACK WORKS

HIGH RECALL
Retrieve widely to get all possible candidates.

HIGH PRECISION
Rerank to surface truly relevant documents.

BETTER CONTEXT
Only the best docs reach the LLM.

BETTER ANSWERS
More accurate, grounded, and trustworthy outputs.

EFFICIENT TRADE-OFF
Extra compute where it matters most.

THE BOTTOM LINE

Retrieval finds candidates.
Reranking finds the best candidates.

RETRIEVAL IS ABOUT FINDING.
RERANKING IS ABOUT CHOOSING.